Cloudboosta Academy

AWS Cloud DevOps Training

CI/CD Pipeline with Azure DevOps, GitOps, and AKS

Full Project Documentation — Build Log, Steps, Errors, Fixes, and Security Enhancements

Group 2

Wisdom Chiemela Uwaga

Bolarinwa Gbenga Akinde

Ayobami Edun

Osahon Igbogbo

Emmanuel Ohuoha

May 2026

Table of Contents

1. Project Overview

The project brief required building a fully automated CI/CD pipeline to deploy three microservices to Azure Kubernetes Service (AKS) using a GitOps model. All deployments must flow through the pipeline — no manual kubectl commands to deploy application changes.

1.1 Project Requirements

- CI/CD pipeline using Azure DevOps Pipelines
- Three microservices in different languages: Python (Flask), Node.js (Express), .NET (ASP.NET Core)
- Container images stored in Azure Container Registry (ACR)
- GitOps deployment model using ArgoCD
- Kubernetes cluster: AKS with minimum 3 nodes
- Security gates: secret scanning, IaC scanning, SAST, CVE scanning, DAST
- Secrets management using Azure Key Vault
- Self-hosted Azure DevOps agent
- Monitoring: Prometheus and Grafana
- Infrastructure as Code using Terraform
- All configuration stored in Git — reproducible from scratch

1.2 Architecture Summary

A developer pushes code to GitHub. Azure DevOps detects the push via webhook and runs the CI pipeline. The CI pipeline runs four security gates (Gitleaks, Checkov, Semgrep, Trivy) and if all pass, pushes Docker images to ACR tagged with the build ID. The CD pipeline updates Kubernetes deployment manifests with the new image tag. ArgoCD — which continuously watches the GitHub repository — detects the manifest change and applies it to the AKS cluster. A DAST scan runs against the live endpoints after deployment.

The key design principle is the GitOps pull model: the pipeline never runs kubectl directly against the cluster. Instead it commits a change to Git, and ArgoCD reads that change and applies it. Git is always the source of truth.

2. Technology Stack

Tool / Service	Role
Azure DevOps Pipelines	CI/CD orchestration — build, scan, push, deploy
GitHub	Source of truth for code and Kubernetes manifests
Azure Container Registry (ACR)	Private Docker image storage
Azure Kubernetes Service (AKS)	Managed Kubernetes cluster — 3 ARM64 nodes
ArgoCD	GitOps continuous delivery engine — watches Git, applies changes
Azure Key Vault	Secrets management — passwords and tokens at runtime
Terraform	Infrastructure as Code — provisions AKS, ACR, Key Vault, agent VM, monitoring
Prometheus + Grafana	Cluster and application metrics, dashboards, alerting
Gitleaks	Secret scanning — blocks pipeline if credentials committed to Git
Checkov	IaC misconfiguration scanning — blocks on security policy violations
Semgrep	SAST — static source code security analysis
Trivy	Container CVE scanning — blocks on CRITICAL/HIGH vulnerabilities
OWASP ZAP	DAST — dynamic security testing against live endpoints
NGINX Ingress	Path-based routing, TLS termination, force-HTTPS redirect
cert-manager	Automated TLS certificate provisioning via Let's Encrypt

3. Infrastructure Setup

Infrastructure is provisioned via Terraform (recommended — see Section 13) or manually via Azure CLI. The manual commands are documented here for reference and for steps not covered by Terraform.

3.1 Azure Login and Subscription

```
az login
az account set --subscription "<SUBSCRIPTION_ID>"
```

3.2 Create Resource Group

```
az group create --name rg-gitops-project --location uksouth
```

3.3 Create Azure Container Registry (ACR)

```
az acr create \
  --resource-group rg-gitops-project \
  --name acrgitopsproject \
  --sku Standard
```

SKU: Standard (not Basic) to support geo-replication if needed. Admin user disabled — AKS pulls via managed identity.

3.4 Create AKS Cluster

```
az aks create \
  --resource-group rg-gitops-project \
  --name aks-gitops-cluster \
  --node-count 3 \
  --node-vm-size Standard_D2ps_v6 \
  --enable-addons monitoring \
  --generate-ssh-keys \
  --attach-acr acrgitopsproject
```

Standard_D2ps_v6 is an ARM64 (Ampere) VM. It costs approximately 50% less than equivalent x86-64 SKUs and enables native linux/arm64 Docker builds on the agent VM without QEMU emulation.

3.5 Connect kubectl to the Cluster

```
az aks get-credentials \
  --resource-group rg-gitops-project \
  --name aks-gitops-cluster
kubectl get nodes
```

3.6 Create Azure Key Vault

```
az keyvault create \
  --name kv-gitops-project \
  --resource-group rg-gitops-project \
```

```
    --location uksouth
```

4. ArgoCD — GitOps Engine

ArgoCD is the GitOps engine. It watches the k8s/ directory in the GitHub repository and reconciles the cluster state with what Git says. If someone manually changes a resource in the cluster, ArgoCD reverts it within 3 minutes. This enforces Git as the single source of truth.

4.1 Install ArgoCD

```
kubectl create namespace argocd
kubectl apply -n argocd -f
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/
install.yaml
kubectl wait --for=condition=ready pod --all -n argocd --timeout=300s
```

4.2 Expose ArgoCD Web UI

```
kubectl patch svc argocd-server -n argocd \
  -p '{"spec": {"type": "LoadBalancer"}}'
```

Retrieve admin password and store in Key Vault:

```
ARGOCD_PASSWORD=$(kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d)
az keyvault secret set --vault-name kv-gitops-project \
  --name argocd-admin-password --value "$ARGOCD_PASSWORD"
```

4.3 ArgoCD AppProject and Application

The AppProject restricts ArgoCD to the microservices namespace only and pins the source to the authorised repository. Wildcard sourceRepos are not permitted.

```
kubectl apply -f k8s/bootstrap/appproject.yaml
kubectl apply -f argocd/application.yaml
```

5. Kubernetes Security Configuration

Every pod runs with a hardened security context. No pod has more privilege than it needs to serve HTTP traffic. Security controls are applied at both the pod level and the namespace level.

5.1 Container Security Context

Applied to every container in every deployment:

Setting	Value	Purpose
allowPrivilegeEscalation	false	Prevents setuid/setgid privilege escalation
readOnlyRootFilesystem	true	Any write outside mounted volumes fails — limits persistence if compromised
capabilities.drop	["ALL"]	Container starts with zero Linux capabilities
seccompProfile.type	RuntimeDefault	Applies runtime default syscall filter (~45 dangerous syscalls blocked)

5.2 Pod Security Context

Setting	Value
runAsNonRoot	true — Kubernetes rejects any image that starts as UID 0
runAsUser	1001 (python-app, dotnet-app) or 1000 (nodejs-app, matches node:20-alpine user)
automountServiceAccountToken	false — pods have no K8s API access by default

5.3 NetworkPolicy (Default Deny)

A default-deny-all policy blocks all ingress and egress for every pod in the microservices namespace. Per-service policies then explicitly re-open only what is needed: ingress from the ingress-nginx namespace and from the monitoring namespace for Prometheus scraping. No other traffic is permitted.

5.4 PodDisruptionBudget

Each service has a PodDisruptionBudget with minAvailable: 1 to ensure at least one replica remains available during node maintenance or rolling updates.

5.5 HorizontalPodAutoscaler

Each service has an HPA with targetCPUUtilizationPercentage: 70 and maxReplicas: 5. The HPA scales out before CPU throttling occurs.

5.6 ResourceQuota and LimitRange

The microservices namespace has a ResourceQuota capping total CPU and memory, and a LimitRange setting default limits per container. This prevents any single workload from consuming all cluster resources.

6. CI Pipeline — Security Gates and Build

The CI pipeline runs on every push to main. All four security gates are hard-fail: a finding in any gate blocks the build immediately. There is no continueOnError and no soft-fail — if Gitleaks finds a secret, the build stops. If Trivy finds a CRITICAL CVE, the image is not pushed to ACR.

6.1 Gate 1 — Secret Scanning (Gitleaks)

Gitleaks scans the full git history for committed secrets. Exit code 1 on findings. The pipeline fails immediately — no image is built.

```
docker run ghcr.io/gitleaks/gitleaks:v8.18.4 detect --source /repo --exit-code
1
```

6.2 Gate 2 — IaC Scanning (Checkov)

Checkov scans the k8s/ directory for Kubernetes security misconfigurations. Findings block the pipeline. SARIF output is published as a pipeline artifact for review.

```
checkov --directory k8s/ --framework kubernetes --output cli
```

6.3 Gate 3 — SAST (Semgrep)

Semgrep scans all microservice source code for insecure patterns. The auto ruleset is used. Findings block the pipeline.

```
semgrep --config=auto microservices/ --sarif
```

6.4 Build — Docker Images (linux/arm64)

All images are built for the linux/arm64 platform to match the AKS node architecture. On an ARM64 build agent, no QEMU emulation is needed. The build agent VM is also Standard_D2ps_v6 (ARM64).

```
docker build --platform linux/arm64 -t python-app:$(Build.BuildId)
microservices/python-app/
```

6.5 Gate 4 — Container CVE Scanning (Trivy)

Trivy scans each built image for known CVEs. Only CRITICAL and HIGH severity unfixed vulnerabilities block the pipeline. Trivy is downloaded at build time and pinned to version 0.50.4. If the download fails, the pipeline fails — there is no bypass.

```
trivy image --exit-code 1 --severity CRITICAL,HIGH --ignore-unfixed python-app:
$(IMAGE_TAG)
```

6.6 Push to ACR

Images are tagged with the immutable build ID (not 'latest') and pushed to ACR. The tag is never reused. This ensures ArgoCD drift detection works correctly and every deployed image is traceable to a specific pipeline run.

7. CD Pipeline — Deploy via GitOps

The CD pipeline triggers automatically when the CI pipeline completes successfully. It never pushes directly to the Kubernetes API. Instead it commits updated manifest files to GitHub, and ArgoCD applies them to the cluster.

7.1 Stage 1 — Fetch Secrets from Key Vault

The ArgoCD admin password and GitHub PAT are fetched from Key Vault at runtime via the AzureKeyVault@2 task. They are never stored in pipeline variables, environment variables, or YAML files.

7.2 Stage 2 — Update Kubernetes Manifests

The CD pipeline patches the image tag in each deployment manifest using sed. It then commits the change to GitHub with a commit message that includes the build ID, requester name, and branch. The [skip ci] flag prevents the commit from re-triggering the CI pipeline.

7.3 Stage 3 — ArgoCD Sync and Health Gate

The pipeline triggers an immediate ArgoCD sync and waits up to 10 minutes for all pods to reach the Healthy state. If any pod fails to start within the timeout, the pipeline fails. This makes the CD pipeline a real deployment status signal — not just a Git push.

ArgoCD is accessed over HTTPS with --grpc-web transport. The --insecure flag is not used. TLS is enforced on the ArgoCD server.

7.4 Stage 4 — DAST (OWASP ZAP)

A ZAP baseline scan runs against all three Ingress endpoints after deployment. Findings are published as artifacts for review. The DAST stage is soft-fail (continueOnError: true) because ZAP baseline scans produce informational findings that do not block promotion — the team reviews the SARIF report after each deployment.

8. Self-Hosted Pipeline Agent

The Azure DevOps pipelines run on a self-hosted agent VM. Using a self-hosted agent gives full control over the build environment, persistent Docker layer caching, and native ARM64 builds without QEMU.

8.1 Create the Agent VM

Via Terraform (recommended — see Section 13) or manually:

```
az vm create \
  --resource-group rg-gitops-project \
  --name ado-pipeline-agent \
  --image Canonical:0001-com-ubuntu-server-jammy:22_04-lts-arm64:latest \
  --size Standard_D2ps_v6 \
  --admin-username azagent \
  --generate-ssh-keys
```

8.2 Install Tools on the Agent

Required: Docker, Azure CLI, kubectl, Python 3 + pip, Terraform. When provisioned by Terraform, all tools are installed automatically via cloud-init on first boot.

8.3 Register the Agent with Azure DevOps

When provisioned by Terraform, the agent self-registers on first boot. For manual registration:

```
./config.sh --url https://dev.azure.com/<ORG> --auth pat --token <PAT> --pool
Default
sudo ./svc.sh install && sudo ./svc.sh start
```

9. Monitoring — Prometheus and Grafana

The kube-prometheus-stack Helm chart deploys Prometheus, Alertmanager, and Grafana in the monitoring namespace. Prometheus scrapes metrics from all three microservices, all AKS nodes, and the Kubernetes control plane components.

9.1 Install kube-prometheus-stack via Helm

When provisioned by Terraform, the chart is installed automatically via the bootstrap module. Manual installation:

```
helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set grafana.adminPassword="$(az keyvault secret show \
    --vault-name kv-gitops-project --name grafana-admin-password --query value
-o tsv)" \
  --values k8s/monitoring/helm-values.yaml
```

Grafana is deployed as ClusterIP (not LoadBalancer) and is not publicly accessible. Access via kubectl port-forward only.

9.2 Access Grafana

```
kubectl port-forward svc/kube-prometheus-stack-grafana 3000:80 -n monitoring
```

Open http://localhost:3000. Six Azure Monitor alert rules are configured by monitoring/setup-alerts.sh: pod restart count, node CPU, node memory, container CPU limit, container memory limit, and ArgoCD out-of-sync.

10. Errors Encountered and How They Were Fixed

This section documents every significant error encountered during the project build, with the root cause and the fix applied. These errors are preserved so future teams doing this project understand where to expect problems.

10.1 ArgoCD AppProject Blocking PDB, HPA, and Ingress

Error: ArgoCD refused to sync PodDisruptionBudget, HorizontalPodAutoscaler, and Ingress resources. Root cause: the AppProject namespaceResourceWhitelist did not include these resource types. Fix: added policy/PodDisruptionBudget, autoscaling/HorizontalPodAutoscaler, and networking.k8s.io/Ingress to the whitelist in argocd/project.yaml.

10.2 nodejs-app Image Tag Missing from ACR

Error: Pod stuck in ImagePullBackOff after CD pipeline ran. Root cause: the CI pipeline for nodejs-app failed silently due to a build error that was missed because continueOnError was true on the Trivy job. Fix: removed continueOnError from all security gate jobs so failures are immediately visible and the push step does not run on a failed scan.

10.3 monitoring/namespace.yaml Causing ArgoCD OutOfSync

Error: ArgoCD showed OutOfSync after adding the monitoring namespace to the k8s/ directory. Root cause: the monitoring namespace was created by Helm (for kube-prometheus-stack) and ArgoCD tried to manage it, causing a conflict. Fix: moved monitoring-namespace.yaml to k8s/bootstrap/ (not watched by ArgoCD).

10.4 kube-prometheus-stack CPU Scheduling Failure

Error: Prometheus pod stuck in Pending with InsufficientCPU. Root cause: the 3-node cluster had insufficient CPU headroom after all microservice pods were scheduled. Fix: reduced Prometheus CPU request from 500m to 100m in helm-values.yaml and set retention to 12h to reduce memory pressure.

10.5 Standard_B2s and Standard_D2s_v5 VM Unavailable

Error: VM quota exceeded in UK South for the initially chosen VM SKU. Fix: switched to Standard_D2ps_v6 (ARM64). This also resolved the separate issue of needing QEMU for ARM64 image builds — the native ARM64 agent eliminated that entirely.

10.6 ARM64 Ubuntu Image Error

Error: az vm create failed with 'image not found' for the ARM64 Ubuntu SKU. Root cause: the full image reference is required for ARM64. Fix: used Canonical:0001-com-ubuntu-server-jammy:22_04-lts-arm64:latest as the full URN.

10.7 Agent Directory Empty — DNS Resolution Failed

Error: Azure DevOps agent download failed silently. Root cause: the agent VM could not resolve vstsagentpackage.azureedge.net during initial provisioning. Fix: added explicit DNS servers to the VM network configuration and retried after the VM was fully initialised.

10.8 VS30063: You Are Not Authorized

Error: Agent registration failed with authorization error. Root cause: the PAT used for agent registration had insufficient scope — it was missing Agent Pools: Read & manage. Fix: created a new PAT with the correct scope.

10.9 Agent Registered to Wrong Pool

Error: Pipelines queued but no agent picked them up. Root cause: the agent registered to a pool named 'Self-Hosted' but the pipeline YAML referenced 'Default'. Fix: re-registered the agent to the Default pool and confirmed it showed Online in Azure DevOps.

10.10 Build 59 — Old Pipeline YAML Still in ADO

Error: Pipeline ran but used outdated YAML. Root cause: Azure DevOps was using a cached pipeline definition. Fix: triggered a manual pipeline run from the latest commit to force re-reading the YAML.

10.11 pip Not Found on ARM64 Agent (Exit Code 127)

Error: Checkov and Semgrep install steps failed. Root cause: pip3 was not installed on the freshly provisioned Ubuntu ARM64 VM. Fix: added python3-pip to the agent VM package install list.

10.12 QEMU Exit Code 255 on ARM64 Agent

Error: Docker builds failed during QEMU setup. Root cause: the pipeline was attempting to configure QEMU on an ARM64 agent where it is not needed. Fix: wrapped the QEMU setup command in an arch check — if the agent is aarch64, QEMU setup is skipped entirely.

10.13 Trivy Binary Incompatible with ARM64

Error: Trivy scan step failed with 'exec format error'. Root cause: Trivy was downloading the Linux-64bit (x86-64) binary instead of Linux-ARM64. Fix: added architecture detection to the install script and selected the correct Trivy binary based on uname -m.

10.14 continueOnError Not Valid at Stage Level

Error: Pipeline YAML validation failed with 'continueOnError is not a valid property for stage'. Root cause: continueOnError is a job-level property, not a stage-level one. Fix: moved continueOnError to the job definition inside the stage.

10.15 ArgoCD CLI Downloading amd64 Binary on ARM64 Agent

Error: argocd login failed with 'exec format error'. Root cause: the CLI install script hardcoded amd64 for the binary architecture. Fix: added the same architecture detection pattern used for Trivy — if uname -m returns aarch64, download argocd-linux-arm64.

11. End-to-End Test Results

11.1 CI Pipeline — Final Successful Run

All six stages completed successfully:

Stage	Tool	Result
Gate 1	Gitleaks v8.18.4	No secrets found — passed
Gate 2	Checkov 3.2.0	No K8s misconfigurations — passed
Gate 3	Semgrep 1.72.0	No insecure code patterns — passed
Build	Docker (linux/arm64)	All 4 images built successfully
Gate 4	Trivy 0.50.4	No CRITICAL/HIGH CVEs — passed
Push	ACR	All 4 images pushed with build ID tag

11.2 CD Pipeline — Final Successful Run

Stage	Result
Fetch Secrets	argocd-admin-password and github-pat retrieved from Key Vault
Update Manifests	All 4 deployment YAMLs patched with new image tag and committed
ArgoCD Sync	microservices-app synced and reached Healthy within 4 minutes
DAST (ZAP)	Baseline scan completed — findings published as artifacts

11.3 Live Application Verification

All three services responded correctly after deployment:

```
curl https://<INGRESS_IP>/api/students  → 200 OK with student list
curl https://<INGRESS_IP>/api/courses   → 200 OK with course list
curl https://<INGRESS_IP>/api/reports/summary → 200 OK with GPA summary
```

11.4 GitOps Self-Healing Verified

Scaled python-app to 0 replicas manually. ArgoCD detected the drift within 3 minutes and restored the deployment to 2 replicas — confirming that Git is enforced as the single source of truth.

12. Key Learnings

ARM64 architecture requires attention at every tool boundary. Every binary download (Trivy, ArgoCD CLI), every Docker build flag, and every base image reference must account for CPU architecture. A pipeline that works on x86-64 will fail in several non-obvious ways on ARM64.

GitOps requires the Git repository to be the single source of truth. Any manual change to the cluster (kubectl scale, kubectl edit) is reverted by ArgoCD. This is a feature — it enforces discipline and makes cluster state auditable.

Security scanning must happen before building, not after. Checking for secrets in source code and misconfigurations in YAML files before building Docker images saves time and prevents wasted pushes to ACR.

Security gates must hard-fail. continueOnError: true on Gitleaks, Checkov, and Semgrep means those gates are decoration — findings are reported but never block the build. Every gate must fail the pipeline on findings.

Architecture decisions need documented trade-offs. Choosing ARM64 over x86-64, ArgoCD over Flux, Terraform over Bicep — each of these has implications that go beyond personal preference. The trade-off matrix is the evidence that choices were deliberate.

Infrastructure as Code is not optional. Every resource created through the Azure portal is a resource that cannot be reproduced reliably. Terraform makes the full environment reproducible in a single command.

13. Infrastructure as Code — Terraform

All Azure infrastructure is defined in Terraform under the terraform/ directory. A single terraform apply provisions everything required to run the project from scratch, including the agent VM which self-registers to Azure DevOps on first boot.

13.1 Module Structure

Module	Resources Provisioned
modules/aks	AKS cluster (3 nodes, Standard_D2ps_v6 ARM64), Secrets Store CSI driver, Key Vault role assignment
modules/acr	Azure Container Registry (Standard SKU), AcrPull role for AKS kubelet identity, AcrPush role for pipeline service principal
modules/keyvault	Azure Key Vault, access policies for Terraform and pipeline service principal, all 5 project secrets
modules/agent	ARM64 Ubuntu VM, VNet/subnet/NSG, cloud-init that installs all tools and registers the agent
modules/bootstrap	All K8s namespaces with required labels, ingress-nginx, cert-manager, kube-prometheus-stack, ArgoCD, GitHub repo credentials, AppProject, and Application

13.2 First-Time Setup

Step 1: Create the Terraform remote state storage (run once):

```
az group create -n rg-tfstate -l uksouth
az storage account create -n stgitopsstate01 -g rg-tfstate --sku Standard_LRS
az storage container create -n tfstate --account-name stgitopsstate01
```

Step 2: Configure variables and export secrets:

```
cd terraform && cp terraform.tfvars.example terraform.tfvars
export TF_VAR_argocd_admin_password="<strong-password>"
export TF_VAR_github_pat="ghp_xxxxxxxxxxxx"
export TF_VAR_grafana_admin_password="<strong-password>"
export TF_VAR_api_key="$(openssl rand -hex 32)"
export TF_VAR_azdo_pat="<azure-devops-pat>"
```

Step 3: Apply:

```
terraform init && terraform plan && terraform apply
```

13.3 Infra Pipeline

The azure-pipelines/infra-pipeline.yml pipeline runs terraform plan automatically on every push to terraform/. Terraform apply requires a manual approval via the infra-apply Azure DevOps Environment. The pipeline reads all sensitive values from Key Vault at runtime — no secrets are stored in Azure DevOps variables.

13.4 Resource Tagging

Every Terraform-managed resource carries four mandatory tags:

Tag	Value
environment	dev | staging | production (validated at plan time)
managed-by	terraform
deployed-by	Object ID of the principal that ran terraform apply
cost-centre	cloud-training (configurable per environment)

Without consistent tags, Azure Cost Management cannot attribute spend to a project, making cost optimisation impossible.

14. Security Review and Remediation

A full security review was conducted after the initial build. Nine vulnerabilities were identified and remediated. The review covered the CI/CD pipeline, Kubernetes configuration, application code, monitoring exposure, and GitOps configuration.

14.1 Vulnerabilities Found and Fixed

#	Severity	Finding	Fix Applied
1	HIGH	No authentication on any API endpoint — all mutating operations accessible to the internet	Added X-API-Key middleware to all three services. API key stored in Kubernetes Secret sourced from Key Vault. GET and /health are exempt.
2	HIGH	Grafana deployed with default password 'admin' on a public LoadBalancer IP	Password injected from Key Vault at Helm install time. Service type changed to ClusterIP — Grafana is no longer publicly accessible.
3	HIGH	ArgoCD admin panel accessible over plain HTTP with --insecure flag in CD pipeline	Removed --insecure from all argocd CLI commands. ArgoCD now requires TLS. The configmap argocd-cmd-params-cm patched to disable insecure mode.
4	MEDIUM	All three security gates (Gitleaks, Checkov, Semgrep) had continueOnError: true — findings never blocked the pipeline	Removed continueOnError from all three gate jobs. Also removed the exit 0 that force-succeeded Gitleaks and the || true patterns in Checkov and Semgrep.
5	MEDIUM	No TLS on public Ingress — all API traffic transmitted in plaintext	Added ssl-redirect and force-ssl-redirect annotations. Added TLS block with cert-manager ClusterIssuer annotation. HTTP requests are redirected to HTTPS.
6	MEDIUM	Wildcard CORS (allow all origins) on all three services	CORS restricted to ALLOWED_ORIGIN environment variable. Variable sourced from Kubernetes Secret. If not set, CORS is blocked entirely.
7	HIGH	NetworkPolicies had no from: selector — any cluster pod could reach microservice pods directly	Added explicit from: namespaceSelector to all four NetworkPolicies, allowing ingress only from ingress-nginx and monitoring namespaces.
8	HIGH	ArgoCD AppProject had sourceRepos: ['*'] — any git repository could be used as source	Replaced wildcard with the specific authorised repository URL in both argocd/project.yaml and k8s/bootstrap/appproject.yaml.
9	MEDIUM	Trivy CVE gate silently skipped if binary download failed (|| echo fallback)	Removed || echo fallback from all four Trivy install steps and removed if command -v trivy guard from scan steps. Download failure now fails the pipeline.

14.2 Security Posture After Remediation

After all nine fixes, the system satisfies the following baseline security requirements:

- All mutating API operations require a valid X-API-Key header

- No monitoring tooling (Grafana, Prometheus) is publicly accessible
- All traffic between clients and the cluster traverses HTTPS — no plaintext HTTP accepted
- Pod-to-pod traffic is zero-trust — only permitted paths are explicitly opened
- ArgoCD can only deploy from the authorised repository to the authorised namespace
- All four CI security gates hard-fail — no gate can be bypassed by a transient error
- Container images are scanned for CVEs before every push — CRITICAL/HIGH vulnerabilities block deployment

15. Service Level Objectives and Error Budgets

SLOs define the reliability promises made to users. Error budgets define how much unreliability is tolerable before new feature work stops. These are defined before deployment, not in response to incidents.

15.1 Service Level Indicators and Objectives

SLI	Target (SLO)	Measurement	Window
Availability — health endpoint returns 2xx	99.9%	sum(rate(http_requests_total{status=~"2.."})) / sum(rate(http_requests_total))	30-day rolling
Latency p99 — end-to-end response time	< 500ms	histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))	5-minute rolling
Error rate — 5xx as % of total requests	< 1%	sum(rate(http_requests_total{status=~"5.."})) / sum(rate(...))	5-minute rolling
Deployment freshness — git push to healthy pods	< 5 min	ArgoCD sync timestamp vs git commit timestamp	Per deployment
Pipeline success rate	>= 95%	Failed CI/CD runs / total runs	7-day rolling

15.2 Error Budget Policy

Budget consumed	Action required
50% (21.9 min downtime)	Engineering lead notified. New feature work paused until budget recovers above 75%.
100% (43.8 min downtime)	Feature freeze. All effort moves to reliability. Post-mortem required within 48 hours.

15.3 Alert Runbooks

A runbook exists for every configured alert. Engineers are expected to follow the runbook before escalating.

Alert	Runbook	SLO Impact
alert-pod-restart-high	docs/runbooks/pod-restart-high.md	Availability — pods restarting may reduce capacity
alert-node-cpu-high	docs/runbooks/node-cpu-high.md	Latency — CPU throttling increases p99
alert-node-memory-high	docs/runbooks/node-memory-high.md	Availability — OOMKill terminates pods
alert-argocd-out-of-sync	docs/runbooks/argocd-out-of-sync.md	Freshness — new versions not deploying

Alert	Runbook	SLO Impact
5xx error rate > 0.5%	docs/runbooks/high-error-rate.md	Error rate SLO directly
Pipeline failure	docs/runbooks/pipeline-failure.md	Freshness — deployments blocked

16. Compliance and Governance

This is an academic project. No regulated data (PII, payment, health) is processed. The controls below map to CIS Kubernetes Benchmark v1.8 and OWASP CI/CD Security Top 10. A named compliance owner and a formal sign-off are required before this system handles real user data.

16.1 Implemented Controls Summary

Domain	Control	Evidence
IAM	Credentials never in code or pipeline variables	azure-pipelines/cd-pipeline.yml — AzureKeyVault@2 tasks
IAM	Service accounts follow least privilege	k8s/rbac/*.yaml; terraform/modules/acr/main.tf
IAM	No auto-mounted K8s service account tokens	k8s/deployments/*.yaml — automountServiceAccountToken: false
IAM	ArgoCD scoped to authorised namespace and repo	argocd/project.yaml — destinations and sourceRepos
Container	Non-root user, read-only filesystem, no capabilities	k8s/deployments/*.yaml — securityContext blocks
Container	CVE scanned before every push	azure-pipelines/ci-pipeline.yml — Trivy hard-fail
Container	Immutable image tags (build ID, never latest)	IMAGE_TAG = Build.BuildId in ci-pipeline.yml
Network	Default-deny NetworkPolicy	k8s/network-policies/default-deny-all.yaml
Network	Per-service ingress restricted to known namespaces	k8s/network-policies/*-netpol.yaml — from: namespaceSelector
Network	TLS enforced on public Ingress	k8s/ingress/ingress.yaml — ssl-redirect annotations
Pipeline	All security gates hard-fail	ci-pipeline.yml — Gitleaks, Checkov, Semgrep, Trivy
Audit	Every deployment traceable to commit, build ID, requester	cd-pipeline.yml — git commit message format
Audit	Secret access logged in Azure Monitor	Azure Key Vault diagnostic settings

16.2 Residual Risks Accepted

Risk	Accepted Because	Required Before Production
No WAF or L7 rate limiting	No real users; no sensitive data	Azure Application Gateway WAF
Shared ArgoCD admin account	Academic project; team is small	Azure AD OIDC SSO for ArgoCD

Risk	Accepted Because	Required Before Production
No automated secret rotation	Manual rotation documented in DR plan	Key Vault rotation policy
Single-region deployment	Cross-region costs ~2x; out of scope	Multi-region AKS with Traffic Manager
No penetration test	STRIDE model + ZAP DAST substitute	Third-party pen test before handling PII

— End of Document —